

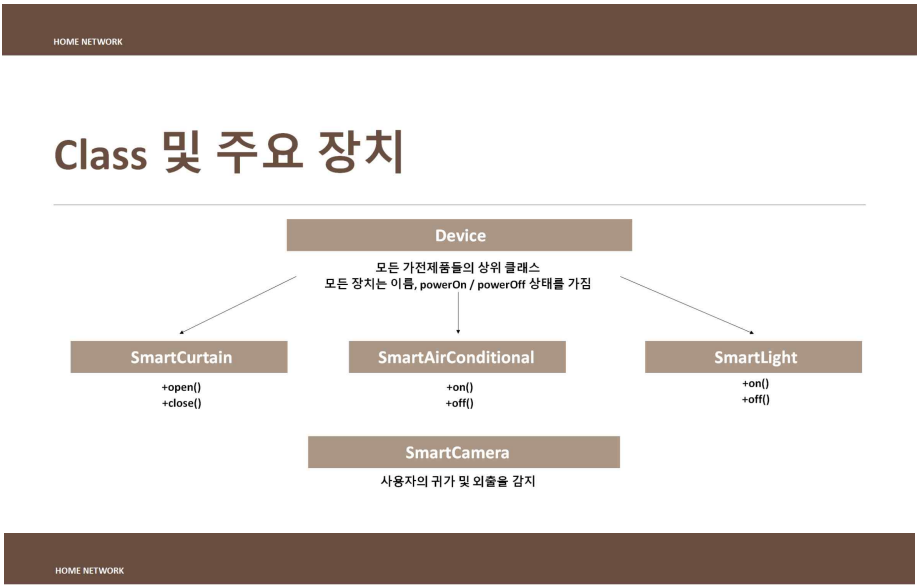
프로젝트 보고서 양식

팀명: 곤고
팀원: 이대성 (202104027), 김태운 (202104009), 김민재 (202004006)

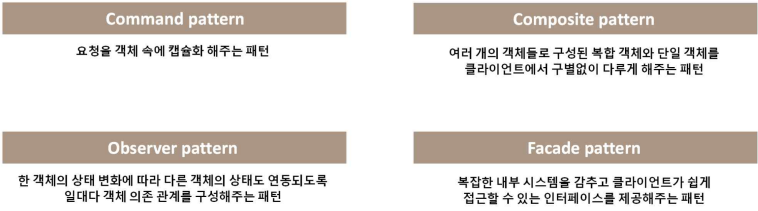
1. 프로젝트 개요

이 프로젝트는 디자인 패턴을 활용하여 객체지향 스마트 홈 네트워크를 설계하는 것이다. 해당 프로젝트에서는 Command, Composite, Observer 패턴을 활용해 스마트 홈 시스템을 설계하여 복잡한 장치 제어 과정을 단순화한다. 여러 가전제품을 연결하여 하나의 기기(ex. 스마트폰)로 편리하게 제어할 수 있도록 설계한다.

2. 객체 지향 설계



02 사용된 패턴들



Device는 모든 가전제품의 상위 클래스로서 모든 장치는 이름 및 powerOn / powerOff의

상태를 상속받는다. SmartCurtain은 커튼을 열고 닫을 수 있는 open()과 close() 메소드가 있다. SmartAirConditional과 Smartright는 전원을 켜고 끄는 on()과 off() 메소드가 있다. 해당 가전제품과는 별개로 SmartCamera가 있으며 사용자의 귀가 및 외출을 감지한다.

사용 패턴

- Command pattern
- Composite pattern
- Observer pattern
- Facade pattern

3. 구현 코드

```
/* ===== 1. 공통 Device ===== */
class Device {                                // 모든 장치의 부모 클래스
  constructor(name){                          // 이름을 받아 초기화
    this.name = name;                        // 장치 고유 이름 보관
    this.isPowered = false;                 // 전원 상태 플래그 초기화
  }
  powerOn(){ this.isPowered = true; }        // 전원 ON
  powerOff(){ this.isPowered = false; }      // 전원 OFF
}
```

- ▶ Device는 모든 가전제품의 상위 클래스로서 모든 장치는 이름 및 powerOn / powerOff의 상태를 상속받는다.

```
/* ===== 2. Command 인터페이스 ===== */
class Command {
  execute(){ throw new Error('execute 필요'); }
}
```

- ▶ Command를 상속하는 장치는 execute 구현을 통해 장치의 상태를 쉽게 제어한다.

```
/* ===== 3-A. 조명 ===== */
class SmartLight extends Device {
  on(){ this.powerOn(); console.log(this.name+' : 조명 ON'); }
  off(){ this.powerOff(); console.log(this.name+' : 조명 OFF'); }
}
class LightOnCommand extends Command {
  constructor(l){ super(); this.l = l; }
  execute(){ this.l.on(); }
}
class LightOffCommand extends Command {
```

```

    constructor(l){ super(); this.l = l; }
    execute(){ this.l.off(); }
}

/* ===== 3-B. 커튼 ===== */
class SmartCurtain extends Device {
    open(){ this.powerOn(); console.log(this.name+' : 커튼 OPEN'); }
    close(){ this.powerOff(); console.log(this.name+' : 커튼 CLOSE'); }
}

class CurtainOpenCommand extends Command {
    constructor(c){ super(); this.c = c; }
    execute(){ this.c.open(); }
}

class CurtainCloseCommand extends Command {
    constructor(c){ super(); this.c = c; }
    execute(){ this.c.close(); }
}

/* ===== 3-C. 에어컨 ===== */

```

```

class SmartAirConditioner extends Device {
    on(){ this.powerOn(); console.log(this.name+' : 에어컨 ON'); }
    off(){ this.powerOff(); console.log(this.name+' : 에어컨 OFF'); }
}

class AirconOnCommand extends Command {
    constructor(a){ super(); this.a = a; }
    execute(){ this.a.on(); }
}

class AirconOffCommand extends Command {
    constructor(a){ super(); this.a = a; }
    execute(){ this.a.off(); }
}

```

- ▶ 각각의 가전제품들은 powerOn / powerOff의 상태를 가진다. SmartLight 클래스로 예를 들자면, SmartLight 클래스의 on()메소드와 off() 메소드를 각각 'LightOnCommand', 'LightOffCommand'로 캡슐화해 장치의 상태 변화를 간단한 명령만으로 실행할 수 있게 한다.

```

/* ===== 4. Composite ===== */
class MacroCommand extends Command {
    constructor(...cmds){ super(); this.cmds = cmds; }
    execute(){ this.cmds.forEach(c => c.execute()); }
}

...

class HomeFacade {
    static build(){

```

```

const light    = new SmartLight('거실 조명');
const curtain = new SmartCurtain('거실 커튼');
const ac       = new SmartAirConditioner('거실 에어컨');
const onLight  = new LightOnCommand(light);
const openCurt = new CurtainOpenCommand(curtain);
const offAc    = new AirconOffCommand(ac);
const offLight = new LightOffCommand(light);
const closeCurt = new CurtainCloseCommand(curtain);
const onAc     = new AirconOnCommand(ac);

```

```

const morningMacro = new MacroCommand(onLight, openCurt, offAc);
const sleepMacro   = new MacroCommand(offLight, closeCurt, onAc);
const leavingMacro  = new MacroCommand(offLight, closeCurt, offAc);
const comingMacro   = new MacroCommand(onLight, openCurt, onAc);

```

- ▶ 여러 명령들을 하나로 묶어 순차적으로 처리한다. 'moningMacro'를 예로 들자면 'moningMacro'의 각 명령들인 LightOnCommand, CurtainOpenCommand, AirconOffCommand는 모두 Command 추상 타입을 상속하였으며, 'moningMacro'는 각 Command를 컴포지션 함으로써, Macro를 실행할 명령들을 장착한다. moningMacro 객체를 통해 장착한 세 가지의 명령이 순차적으로 실행된다.

```

/* ===== 5. Invoker ===== */
class Button {
  constructor(label, cmd) {
    this.label = label; // 버튼의 이름 저장
    this.cmd = cmd;     // 버튼을 눌렀을 때 실행할 명령 저장
  }
  press(){ // 사용자가 버튼을 눌렀을 경우
    console.log(['+this.label+' 버튼']);
    this.cmd.execute(); // Command 객체의 excute() 메서드 실행
    console.log("");
  }
}

```

- ▶ Button 클래스는 사용자의 요청을 받아 .excute()를 호출하여 커맨드를 수행해주는 중간자 역할이다.

```

/* ===== 6-A. 카메라 Subject ===== */
class SmartCamera {
  constructor(name){ this.name =name; this.observers = []; }
  subscribe(o){ this.observers.push(o); }
  notify(e){ this.observers.forEach(o=>o.update(e)); }

  detectArrival(){
    console.log(this.name+' : 귀가 감지');
    this.notify({ type:'arrival' });
  }
  detectDeparture(){
    console.log(this.name+' : 외출 감지');
  }
}

```

```

        this.notify({ type:'departure' });
    }
}
/* ===== 6-B. Observer ===== */
class MacroRunner {
    constructor(evtType, macro){
        this.evtType = evtType; this.macro = macro;
    }
    update(e){
        if(e.type === this.evtType){
            console.log('자동화 : '+e.type+' → 매크로 실행');
            this.macro.execute();
        }
    }
}

```

- ▶ 카메라가 사용자의 귀가 및 외출을 감지했을 경우 감지한 이벤트에 따른 상태 변화를 중계하고 설정된 명령을 실행한다. 'cam' 이라는 'SmartCamera' 객체 생성, 옵저버로 evtType : "arrival", macro : 'comingMacro'를 등록한다. 귀가를 예로 들자면, 귀가를 감지했을 경우 notify()를 호출, 등록된 Observer에게 'arrival' 이벤트를 전달한다. evtType === "arrival"인 Observer만 반응하며, comingMacro를 실행한다.

```

/* ===== 7. Facade ===== */
class HomeFacade {
    static build(){
        const light    = new SmartLight('거실 조명');
        const curtain  = new SmartCurtain('거실 커튼');
        const ac       = new SmartAirConditioner('거실 에어컨');
        const onLight  = new LightOnCommand(light);
        const openCurt = new CurtainOpenCommand(curtain);
        const offAc    = new AirconOffCommand(ac);
        const offLight = new LightOffCommand(light);
        const closeCurt = new CurtainCloseCommand(curtain);
        const onAc     = new AirconOnCommand(ac);

        const morningMacro = new MacroCommand(onLight, openCurt, offAc);
        const sleepMacro   = new MacroCommand(offLight, closeCurt, onAc);
        const leavingMacro  = new MacroCommand(offLight, closeCurt, offAc);
        const comingMacro   = new MacroCommand(onLight, openCurt, onAc);
        const morningBtn   = new Button('Morning', morningMacro);
        const sleepBtn     = new Button('Sleep', sleepMacro);
        const leavingBtn   = new Button('LeavingHome', leavingMacro);
        const comingBtn    = new Button('ComingHome', comingMacro);
        const cam = new SmartCamera('현관 카메라');
        cam.subscribe(new MacroRunner('arrival', comingMacro));
        cam.subscribe(new MacroRunner('departure', leavingMacro));
        return { morningBtn, sleepBtn, leavingBtn, comingBtn, cam };
    }
}

```

- ▶ HomeFacade클래스는 여러 장치, 명령, 이벤트감지 등 복잡한 시스템을 단 하나의 build() 메소드 호출만으로 간편하게 구성할 수 있게 한다. HomeFacade.build() 에서는 내부에서 명령 객체 생성, 매크로 명령 생성, 버튼에 명령 연결, 카메라와 Observer 연결

등 많은 작업들을 수행한다. 하지만 외부에서는 이런 복잡한 내부 구조를 알 필요가 없다. 스마트 홈 시스템 전체의 복잡한 초기 설정과 구성을 감추어 줌으로써 사용자는 복잡한 시스템을 단순하게 사용할 수 있게끔 한다.

부록 프로젝트 수행 인증 사진

